

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Tarea 1: Arquitectura de cebolla (arquitectura limpia)

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Aquí va el nombre del Alumno

mail@mail.com

7CM2

26 de agosto de 2026

Índice

1. Introducción	2
2. Conceptos previos	3
2.1. El acoplamiento hacia la base de datos	3
2.2. El principio de inversión de dependencias	3
3. Desarrollo	4
3.1. Origen y formulaciones	4
3.2. Modelo del dominio	5
3.3. Interfaz de usuario	6
3.4. Infraestructura	6
3.5. Pruebas	7
4. Análisis comparativo	8
4.1. Tres nombres para la misma regla	8
4.2. Costos y críticas	8
5. Conclusión	9
6. Referencias Bibliográficas	10

Índice de figuras

1. Anillos de la arquitectura de cebolla y dirección de las dependencias 4

Índice de tablas

1. Comparación entre las tres arquitecturas centradas en el dominio 8

1. Introducción

La unidad temática I de esta unidad de aprendizaje ordena las arquitecturas de desarrollo web según un mismo criterio: dónde se coloca la lógica de negocio y de qué depende esa lógica. El apartado 1.4 corresponde a la arquitectura de cebolla, conocida también como arquitectura limpia. Este documento reúne su origen, la regla que la define y los cuatro elementos que el temario enumera: modelo del dominio, interfaz de usuario, infraestructura y pruebas.

El punto de partida no es una preferencia estética. Es un problema medible de la arquitectura tradicional de n capas, que el apartado anterior del temario ya introduce, y que consiste en que la lógica de negocio termina dependiendo de la base de datos. La arquitectura de cebolla invierte esa dependencia. El resto del documento explica con qué mecanismo lo hace, qué se gana, qué cuesta y en qué se diferencia de las otras dos formulaciones de la misma idea que circulan en la industria.

2. Conceptos previos

2.1. El acoplamiento hacia la base de datos

En el esquema clásico de tres capas la interfaz de usuario llama a la capa de lógica de negocio, y ésta llama a la capa de acceso a datos. Cada capa depende de la que tiene debajo. Por transitividad, la lógica de negocio depende de la persistencia y la aplicación completa queda construida encima del esquema relacional.

Palermo describió la consecuencia en 2008 [1]: cambiar el motor de almacenamiento, el ORM o el nombre de una columna obliga a tocar código de negocio que no tiene relación con esa decisión técnica. La base de datos, que es un detalle de almacenamiento, acaba siendo el cimiento del sistema.

2.2. El principio de inversión de dependencias

El principio de inversión de dependencias, la letra D de SOLID, enuncia que los módulos de alto nivel no deben depender de los de bajo nivel y que ambos deben depender de abstracciones. Aplicado a un par de clases, el principio produce una interfaz entre quien llama y quien ejecuta. Aplicado a la totalidad del sistema, produce la arquitectura de cebolla.

Conviene separar dos cosas que suelen confundirse. La dependencia de código fuente es qué archivo necesita conocer a qué otro para compilar. El flujo de control es qué método llama a qué método en tiempo de ejecución. La inversión de dependencias consiste precisamente en hacer que ambos apunten en direcciones opuestas.

3. Desarrollo

3.1. Origen y formulaciones

Jeffrey Palermo publicó en julio de 2008 la serie de artículos que dio nombre a la arquitectura de cebolla [1]. Su propuesta se resume en cuatro postulados:

- La aplicación se construye alrededor de un modelo de objetos independiente.
- Las capas internas definen interfaces y las externas las implementan.
- La dirección del acoplamiento apunta hacia el centro.
- Todo el código del núcleo puede compilarse y ejecutarse por separado de la infraestructura.

Cuatro años después, Robert C. Martin publicó el artículo que popularizó el nombre de arquitectura limpia [2] y en 2017 lo desarrolló en un libro [3]. Su enunciado central es la regla de dependencia: las dependencias del código fuente «*can only point inwards*». Nada de un círculo interior puede nombrar algo de un círculo exterior, y la restricción alcanza a clases, funciones, variables y formatos de datos.

Los dos autores dibujan lo mismo con distinta nomenclatura. Palermo nombra sus anillos modelo del dominio, servicios de dominio, servicios de aplicación e infraestructura. Martin los nombra entidades, casos de uso, adaptadores de interfaz y frameworks. La figura 1 representa la versión de Palermo, que es la que sigue el temario.

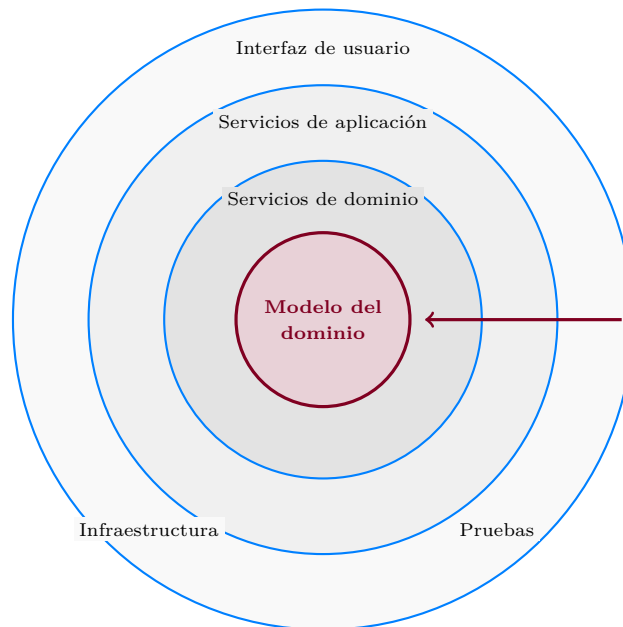


Figura 1: Anillos de la arquitectura de cebolla y dirección de las dependencias

3.2. Modelo del dominio

El modelo del dominio ocupa el centro. Contiene las entidades y los objetos de valor que representan las reglas del negocio, aquellas que seguirían siendo ciertas aunque la aplicación no existiera como software. Un pedido que sabe calcular su total a partir de sus líneas. Una cuenta que rechaza cualquier operación que la deje por debajo del saldo mínimo. El vocabulario y la técnica provienen del diseño guiado por el dominio de Evans [4].

En Java esto significa clases sin anotaciones de framework. Sin `@Entity`, sin `@Component`, sin importaciones de Spring ni de JPA. La restricción tiene una comprobación objetiva: si el módulo del dominio compila con un descriptor de construcción que no declara ninguna dependencia de infraestructura, el núcleo está aislado [5]. Si no compila, hay una fuga que localizar.

Alrededor del modelo, Palermo coloca los servicios de dominio, que resuelven operaciones de negocio que no pertenecen a una sola entidad, y el primer anillo de interfaces: los repositorios. Un repositorio se declara dentro del núcleo como interfaz y se implementa fuera de él. El núcleo enuncia qué necesita y nunca cómo se satisface. El código 1 muestra esa separación.

Código 1: Interfaz declarada en el núcleo y caso de uso que la consume

```
1 // Nucleo: la interfaz vive junto al dominio
2 public interface PedidoRepository {
3     Optional<Pedido> buscarPorId(PedidoId id);
4     void guardar(Pedido pedido);
5 }
6
7 // Caso de uso: depende de la abstraccion, no del ORM
8 public class ConfirmarPedido {
9
10     private final PedidoRepository repositorio;
11
12     public ConfirmarPedido(PedidoRepository repositorio) {
13         this.repositorio = repositorio;
14     }
15
16     public void ejecutar(PedidoId id) {
17         Pedido pedido = repositorio.buscarPorId(id)
18             .orElseThrow(PedidoNoEncontrado::new);
19         pedido.confirmar();
20         repositorio.guardar(pedido);
21     }
22 }
```

Ninguna de las dos clases menciona una tecnología de persistencia. La clase `ConfirmarPedido` corresponde al anillo de servicios de aplicación en la terminología de Palermo y al anillo de casos de uso en la de Martin. Orquesta entidades para cumplir un objetivo del usuario y decide la secuencia de pasos, sin saber de dónde llegan los datos ni dónde se guardan.

3.3. Interfaz de usuario

La interfaz de usuario se sitúa en el anillo exterior, al mismo nivel que la infraestructura y las pruebas. Es un mecanismo de entrega y por lo tanto es reemplazable: la misma aplicación puede exponerse como sitio web, como API consumida por un cliente de escritorio o como línea de comandos, sin que el núcleo cambie.

En la pila que usa el curso, un cliente Angular consume una API REST publicada por controladores de Spring. Ambas piezas viven en el anillo exterior. El controlador recibe la petición HTTP, la traduce a una llamada al caso de uso y convierte el resultado en JSON. El dominio no sabe que HTTP existe.

Martin llama a este anillo adaptadores de interfaz y le asigna los controladores, los presentadores y los conversores de formato [2]. De ahí sale un criterio práctico para ubicar una clase: si menciona `HttpServletRequest`, `ResponseEntity` o un DTO pensado para serializarse en JSON, pertenece al anillo exterior y no al núcleo.

La dirección del acoplamiento se conserva aunque el usuario sea quien inicia la interacción. El controlador conoce y llama al caso de uso. El caso de uso no conoce al controlador.

3.4. Infraestructura

El anillo de infraestructura agrupa todo lo que comunica a la aplicación con el exterior: persistencia, mensajería, correo, clientes HTTP hacia servicios de terceros, sistema de archivos y relojes del sistema. Su trabajo es implementar las interfaces que el núcleo declaró.

Aquí se observa el mecanismo completo de la inversión. En tiempo de compilación, el módulo de infraestructura depende del núcleo, porque necesita conocer la interfaz `PedidoRepository` para poder implementarla. En tiempo de ejecución el flujo de control va en sentido contrario: el caso de uso invoca al repositorio y termina ejecutando código de JPA. Las dos flechas apuntan en direcciones opuestas y eso es lo que consigue el principio de inversión de dependencias.

El ensamblado ocurre en un solo punto, la raíz de composición. En Spring corresponde a una clase de configuración o al escaneo de componentes, y es el único lugar del sistema donde se nombran a la vez la abstracción y la implementación concreta que la satisface [5].

La consecuencia práctica es acotada y verificable. Sustituir PostgreSQL por MongoDB modifica el módulo de infraestructura y la configuración. El dominio y los casos de uso no se recompilan por ese cambio, y sus pruebas siguen pasando sin tocarlas.

3.5. Pruebas

El diagrama original de Palermo coloca las pruebas en el anillo exterior, junto a la interfaz de usuario y la infraestructura [1]. La ubicación tiene una razón: una prueba es otro cliente del núcleo. Ocupa el mismo lugar que la interfaz de usuario y accede al sistema por las mismas fronteras.

Como el núcleo no depende de frameworks, sus pruebas son unitarias en sentido estricto. No hacen falta base de datos, contexto de Spring ni servidor de aplicaciones. Se construye la entidad, se invoca el método y se verifica el invariante. Esa es la razón por la que la testabilidad aparece siempre entre los objetivos declarados de la arquitectura [2]: no es un beneficio añadido, es una consecuencia directa de la regla de dependencia.

La distribución de pruebas suele seguir una pirámide por anillos:

- Dominio: la mayor cantidad de pruebas. Verifican invariantes, cálculos y transiciones de estado. Son rápidas porque no tocan nada externo.
- Servicios de aplicación: pruebas de cada caso de uso. Como los repositorios son interfaces del núcleo, se sustituyen por implementaciones en memoria o por dobles de prueba.
- Infraestructura: pruebas de integración contra la base de datos real, habitualmente con contenedores efímeros, para comprobar que el adaptador cumple el contrato que prometió.
- Extremo a extremo: las menos, por costo de mantenimiento y tiempo de ejecución.

Una advertencia documentada por Bogdanović conviene retenerla [6]: los dobles de prueba deben sustituir dependencias externas al núcleo, no al dominio mismo. Cuando se reemplazan entidades por mocks, las pruebas dejan de verificar comportamiento y pasan a verificar llamadas, de modo que cualquier refactorización interna las rompe aunque el sistema siga siendo correcto.

4. Análisis comparativo

4.1. Tres nombres para la misma regla

La arquitectura de cebolla no apareció aislada. Alistair Cockburn había publicado en 2005 la arquitectura hexagonal, también llamada de puertos y adaptadores [7], y el propio Palermo la reconoce como antecedente. La tabla 1 resume las tres formulaciones.

	Hexagonal (2005)	Cebolla (2008)	Limpia (2012)
Autor	Alistair Cockburn	Jeffrey Palermo	Robert C. Martin
Metáfora	Hexágono rodeado de puertos y adaptadores	Anillos concéntricos	Círculos concéntricos
Centro	La aplicación	Modelo del dominio	Entidades
Divisiones internas	No las prescribe	Dominio, servicios de dominio, servicios de aplicación	Entidades, casos de uso, adaptadores, frameworks
Énfasis propio	Simetría entre el lado que entra y el que sale	Disciplina de anillos y servicios de dominio	Casos de uso explícitos y enunciado de la regla de dependencia

Tabla 1: Comparación entre las tres arquitecturas centradas en el dominio

Las tres se apoyan en el principio de inversión de dependencias, ponen la lógica de negocio en el centro, dejan la tecnología en el borde y obtienen testabilidad como resultado [8]. Jovanović, tras compararlas capa por capa, concluye que la estructura de carpetas a la que llegan los tres enfoques es prácticamente la misma y que la discusión sobre cuál adoptar importa menos que aplicar el criterio de forma consistente [9].

4.2. Costos y críticas

El costo principal aparece en el mapeo entre fronteras. Un mismo concepto de negocio suele existir por triplicado: como entidad del dominio, como DTO de entrada o salida y como modelo de persistencia. Cada frontera exige un traductor, y ese código de traducción hay que escribirlo y mantenerlo.

Los señalamientos que recoge la literatura crítica son consistentes entre sí [10]. Interfaces con una sola implementación que nunca tendrán otra. Capas que se limitan a reenviar la llamada a la siguiente sin agregar nada. Código de una misma funcionalidad repartido entre carpetas separadas por criterio técnico, de modo que entender una operación obliga a recorrer varios módulos. A eso se suma la indirección adicional en tiempo de ejecución, que en sistemas sensibles al rendimiento hay que medir en lugar de suponer.

El criterio para decidir no es el tamaño del código sino la complejidad de las reglas de negocio y el tiempo de vida esperado del sistema. Una aplicación que traslada formularios a tablas sin reglas propias no gana nada con cuatro anillos. Un sistema cuyas reglas cambian, con varios equipos trabajando en paralelo y años de mantenimiento por delante, amortiza el costo de la separación.

5. Conclusión

La arquitectura de cebolla aplica el principio de inversión de dependencias a la escala del sistema completo. En lugar de invertir la relación entre dos clases, invierte la relación entre la lógica de negocio y la tecnología que la rodea, de manera que la base de datos, el framework web y el cliente pasan a ser detalles sustituibles en el borde.

El resultado se puede comprobar y no depende de la opinión de quien revisa el código. El núcleo compila sin infraestructura en el classpath, sus pruebas corren sin base de datos y cambiar de motor de almacenamiento afecta a un módulo identificable. Cuando alguna de esas tres afirmaciones deja de ser cierta, la regla de dependencia se rompió en algún punto.

Las formulaciones hexagonal, de cebolla y limpia difieren en nomenclatura y en el grado de detalle con que prescriben los anillos interiores, pero coinciden en la regla y en la estructura resultante. La elección entre ellas pesa menos que la disciplina para sostener la dirección de las dependencias en un equipo completo durante meses.

Para el proyecto del curso la traducción es directa. Las reglas de negocio se escriben en Java sin anotaciones, Spring y JPA quedan confinados al anillo exterior, Angular es un mecanismo de entrega más, y una raíz de composición se encarga de unir las piezas.

Aquí va el nombre del Alumno

6. Referencias Bibliográficas

- [1] Jeffrey Palermo. *The Onion Architecture: part 1*. 2008. URL: <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/> (visitado 26-08-2026).
- [2] Robert C. Martin. *The Clean Architecture*. 2012. URL: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> (visitado 26-08-2026).
- [3] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN: 978-0134494166.
- [4] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. ISBN: 978-0321125217.
- [5] Foojay.io. *Clean Architecture with Spring Boot and MongoDB*. URL: <https://foojay.io/today/clean-architecture-with-spring-boot-and-mongodb/> (visitado 26-08-2026).
- [6] Valentina Bogdanović. *Fragile Unit Tests in Clean Architecture*. URL: <https://journal.optivem.com/p/fragile-unit-tests-in-clean-architecture> (visitado 26-08-2026).
- [7] Alistair Cockburn. *Hexagonal Architecture (Ports and Adapters)*. 2005. URL: <https://alistair.cockburn.us/hexagonal-architecture/> (visitado 26-08-2026).
- [8] Herberto Graça. *Onion Architecture*. 2017. URL: <https://herbertograca.com/2017/09/21/onion-architecture/> (visitado 26-08-2026).
- [9] Milan Jovanović. *Clean Architecture vs Onion Architecture vs Hexagonal Architecture*. URL: <https://milanjovanovic.tech/blog/clean-architecture-vs-onion-vs-hexagonal> (visitado 26-08-2026).
- [10] Three Dots Labs. *Is Clean Architecture Overengineering?* URL: <https://threedots.tech/episode/is-clean-architecture-overengineering/> (visitado 26-08-2026).